Ιόνιο Πανεπιστήμιο – Τμήμα Πληροφορικής

Αρχιτεκτονική Υπολογιστών

Εντολές Διακλάδωσης

(και ο μηχανισμός κλήσης συναρτήσεων)

https://mixstef.github.io/courses/comparch/

Μ.Στεφανιδάκης



Χρήση Διευθύνσεων Μνήμης

- Προσπέλαση δεδομένων (μεταβλητές προγραμμάτων)
  - Εντολές μηχανής που διαβάζουν ή γράφουν δεδομένα σε διευθύνσεις μνήμης
  - Διάφοροι τρόποι σχηματισμού διεύθυνσης μνήμης δεδομένων για την υποβοήθηση των γλωσσών προγραμματισμού
- Ανάκληση εντολών (ροή εκτέλεσης)
  - Αυτόματη ενημέρωση του καταχωρητή program counter (pc) στη διεύθυνση μνήμης της επόμενης προς εκτέλεση εντολής
  - Οι εντολές διακλάδωσης (branches/jumps) αλλάζουν τιμή στον pc: φορτώνουν στον pc μια νέα διεύθυνση μνήμης
    - Όπου θα μεταφερθεί η ροή εκτέλεσης

Τι περιγράφει μια εντολή διακλάδωσης

- Πότε θα «εκτελεστεί» η εντολή διακλάδωσης
  - Μια συνθήκη που πρέπει να ισχύει για να φορτωθεί η νέα διεύθυνση στον pc
    - Αν δεν ισχύει η συνθήκη, ο pc συνεχίζει με τη διεύθυνση της εντολής που ακολουθεί την εντολή διακλάδωσης
  - Η συνθήκη αυτή υπολογίζεται:
    - Είτε στην ίδια την εντολή διακλάδωσης

      `“if a == b then goto X”`
    - Είτε σε προηγούμενη εντολή

      `“compare a and b;`
      `if equal then goto X”`
  - Υπάρχουν και εντολές διακλάδωσης χωρίς συνθήκη

    `“goto X”`

Τι περιγράφει μια εντολή διακλάδωσης (2)

- Πού θα μεταφερθεί η ροή του προγράμματος εάν «εκτελεστεί» η εντολή διακλάδωσης
  - Ποια είναι η νέα διεύθυνση που θα φορτωθεί στον pc
- Η νέα διεύθυνση μπορεί να περιγράφεται:
  - Είτε ως απόλυτη (absolute) διεύθυνση

    `"πήγαινε στη διεύθυνση 1000"` `(pc ← 1000)`

    όπου η διεύθυνση συχνά περιέχεται σε έναν καταχωρητή ή στη μνήμη (indirect)

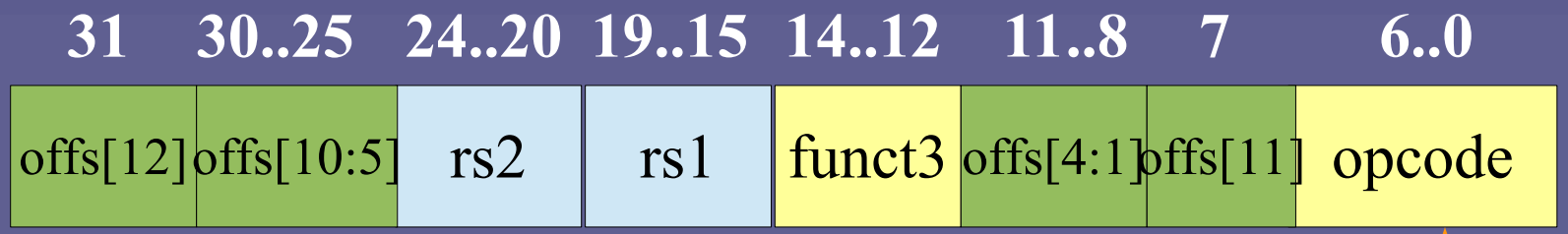
    `"πήγαινε στη διεύθυνση που περιέχει ο R5"`
    `(pc ← [R5])`
  - Είτε ως σχετική (relative) διεύθυνση ως προς την τρέχουσα θέση, δηλ. την τρέχουσα τιμή του PC

    `"πήγαινε 40 bytes πριν την τρέχουσα εντολή"`
    `(pc ← pc - 40)`

RISC-V (32 και 64-bit): Εντολές διακλάδωσης υπό συνθήκη

- Ακολουθείται η τακτική "compare-and-branch"
  - Υπολογισμός συνθήκης και διακλάδωση εκτελούνται στην ίδια εντολή
- Υπολογισμός συνθήκης: σύγκριση τιμών δύο καταχωρητών
  - Μπορούμε να επιλέξουμε σύγκριση αριθμών με ή χωρίς πρόσημο
- Εάν ισχύει η ζητούμενη συνθήκη:
  - Προστίθεται στον pc μια ακέραια σταθερά (offset με πρόσημο) των 13 bits που περιλαμβάνεται στην εντολή
  - Η μετατόπιση μπορεί να είναι έως ±4KB

RISC-V Conditional Branches (B-type)



funct3	σύγκριση (cmp)
000	BEQ (==)
001	BNE (!=)
100	BLT (<)
101	BGE (>=)
110	BLTU (<)
111	BGEU (>=)

είδος σύγκρισης

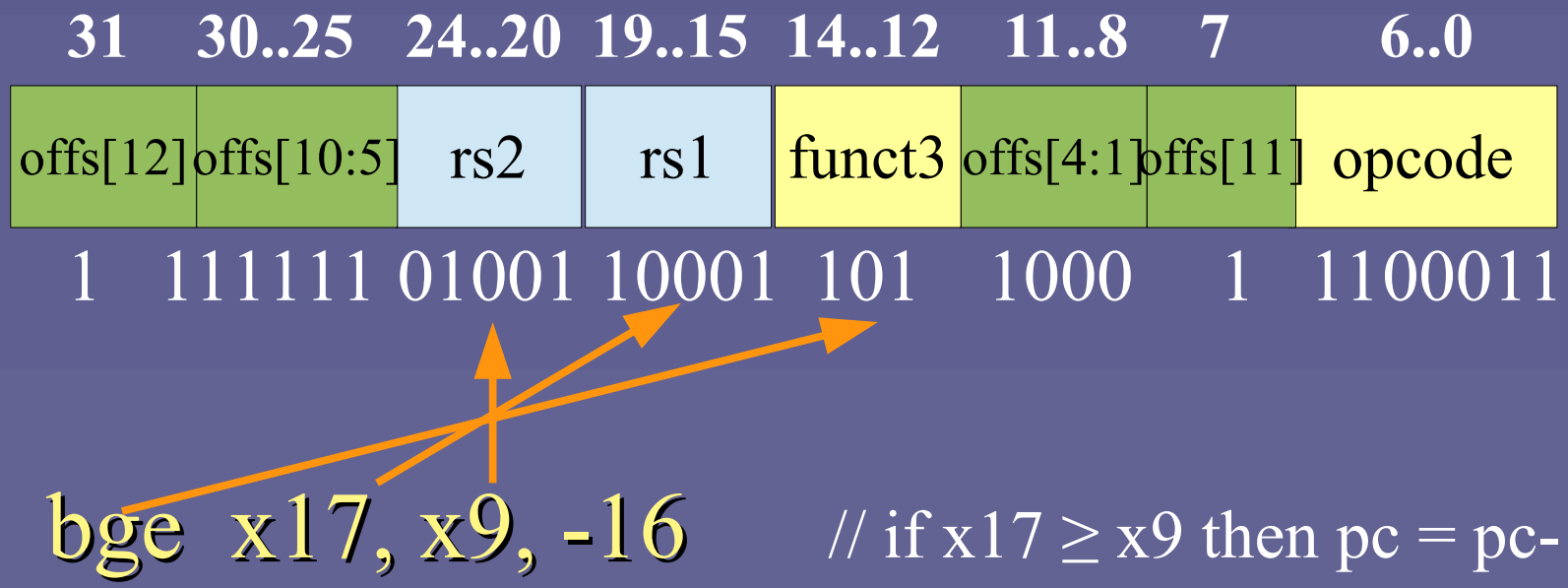
1100011

if rs1 *cmp* rs2 is true:

pc ← pc ± offset

Οι εντολές RISC-V είναι είτε 32 bits είτε (συμπιεσμένες) 16 bits: κάθε εντολή βρίσκεται πάντα σε ζυγή διεύθυνση → το offs[0] είναι πάντα 0 και δεν αποθηκεύεται στην εντολή

Παράδειγμα



bge x17, x9, -16 // if x17 ≥ x9 then pc = pc-16

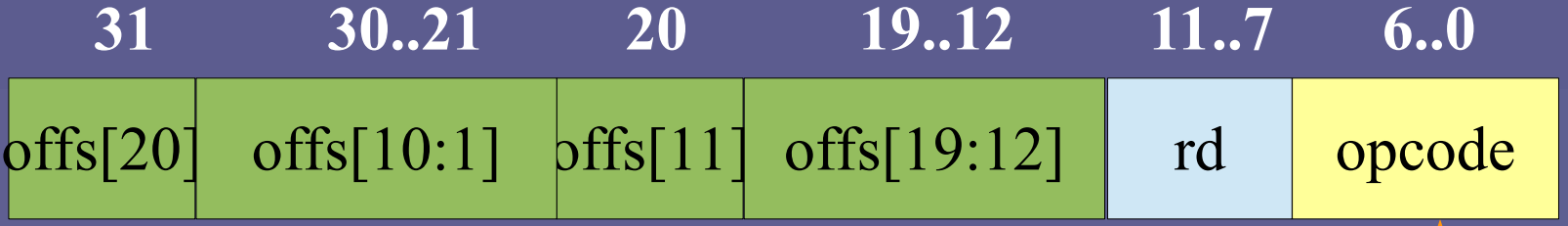
bytes εντολής: FE 98 D8 E3

Σημ.: -16 (σε 13 bits) = 1 1111 1111 000(0)
Δεν ξεχνάμε και το bit0 που θεωρείται πάντα 0

RISC-V (32 και 64-bit): Εντολές διακλάδωσης χωρίς συνθήκη

- Εντολές διακλάδωσης χωρίς συνθήκη
  - PC ← PC + ακέραια σταθερά των 21 bits (μετατόπιση ±1MB)
    - Εντολή JAL (jump and link)
  - PC ← ακέραια σταθερά των 12 bits + το περιεχόμενο ενός καταχωρητή (μετατόπιση σε όλο το εύρος διευθύνσεων)
    - Εντολή JALR (jump and link register)

RISC-V Jump and link (JAL, J-type)



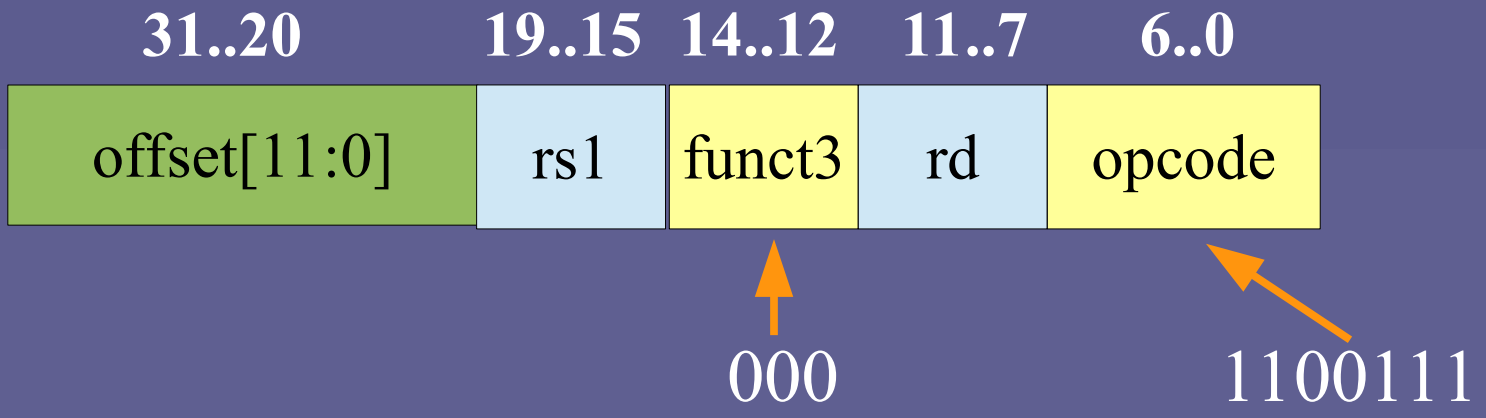
**το offs[0] είναι πάντα 0 και δεν αποθηκεύεται*

rd ← pc + 4 *(next instruction)*

pc ← pc ± offset

- Για διακλάδωση χωρίς συνθήκη σε σχετική (relative) προς τον pc διεύθυνση
  - PC ← PC + ακέραια σταθερά των 21 bits (μετατόπιση ±1MB)
  - Στη θέση του rd χρησιμοποιείται ο x0 (η τιμή pc+4 χάνεται)

RISC-V Jump and link register (JALR, I-type)



rd ← pc + 4 *(next instruction)*

pc ← rs1 ± offset *(bit0 αθροίσματος → 0)*

- Για διακλάδωση χωρίς συνθήκη, η χρήση εξαρτάται από την αρχικοποίηση του rs1
  - Διακλάδωση σε απόλυτη διεύθυνση ή σχετική προς τον pc (αν ο rs1 περιέχει ήδη την τιμή του pc)
  - Στη θέση του rd χρησιμοποιείται ο x0 (η τιμή pc+4 χάνεται)

Εντολές για ανάθεση μεγάλων σταθερών (U-type)



0110111 (lui)
0010111 (auipc)

lui rd, imm // rd ← imm << 12

auipc rd, imm // rd ← pc + (imm << 12)

Εντολές για να βάλουμε σταθερές των 20 bits στο «πάνω» μέρος ενός καταχωρητή. Συνδυάζονται με άλλες εντολές που φορτώνουν τα 12 «χαμηλότερα» bits του καταχωρητή (π.χ. ADDI, αλλά προσοχή στην επέκταση προσήμου!)

Χρήση lui + jalr → διακλάδωση σε απόλυτη διεύθυνση

Χρήση auipc + jalr → διακλάδωση σε σχετική προς τον PC διεύθυνση

x86-64 (64-bit mode): Εντολές διακλάδωσης υπό συνθήκη

- Η συνθήκη υπολογίζεται από προηγούμενη εντολή
  - Χρήση flags για τη μεταφορά του αποτελέσματος της σύγκρισης στην εντολή διακλάδωσης που ακολουθεί
- Εάν ισχύει η ζητούμενη συνθήκη → προστίθεται στον pc μια ακέραια σταθερά (offset με πρόσημο) των 8 ή 32 bits

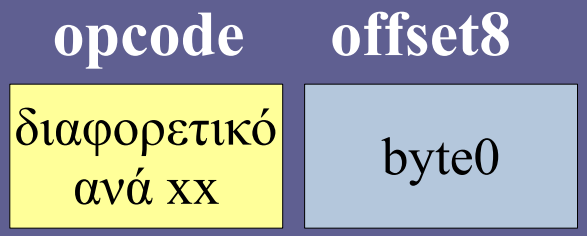
Σημαίες κατάστασης (flags)

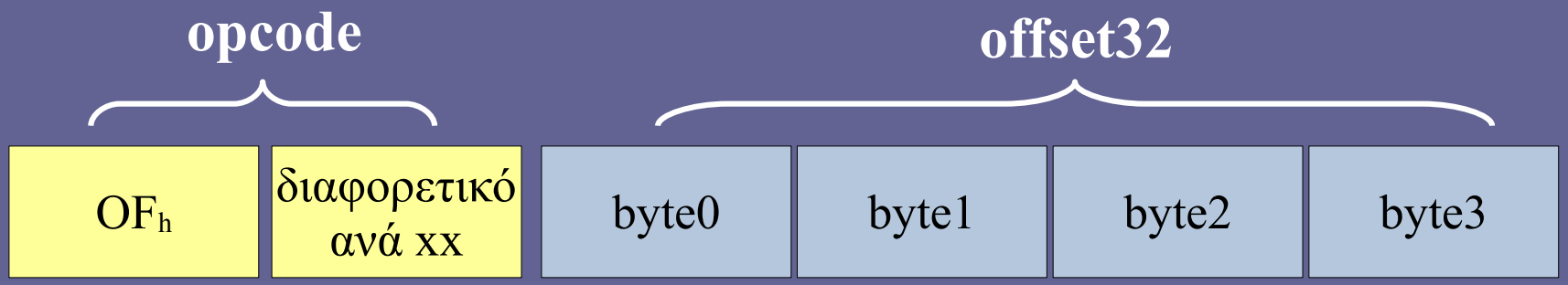
- Σε αρχιτεκτονικές που εκτελούν διακλάδωση υπό συνθήκη με βάση το αποτέλεσμα προηγούμενης εντολής
- Τα flags είναι μια ομάδα bits που αναφέρουν την κατάσταση της μονάδας επεξεργασίας μετά την εκτέλεση μιας εντολής
  - Κάθε εντολή, ανάλογα με τη λειτουργικότητά της, επηρεάζει ορισμένα μόνο flags
- Τα πιο κοινά flags:
  - (Z)ero flag = μηδενικό αποτέλεσμα (της προηγούμενης πράξης)
  - (S)ign flag = αρνητικό αποτέλεσμα
  - (C)arry flag = ύπαρξη τελικού κρατουμένου
  - o(V)erflow flag = ένδειξη υπερχείλισης (αριθμοί με πρόσημο)
- Οι εντολές διακλάδωσης υπό συνθήκη εξετάζουν την τιμή ορισμένων flags για να αποφασίσουν αν θα γίνει διακλάδωση ή όχι

x86-64 Jxx instructions

- Έλεγχος αν κάποιο flag από τα c, z, s, v (CF, ZF, SF, OF στην ορολογία της x86-64) έχει τεθεί ή όχι
  - JC/JNC, JZ/JNZ, JS/JNS, JO/JNO
- Έλεγχος ισότητας δύο αριθμών
  - JE (z = 1)/JNE (z = 0)
- Αποτέλεσμα σύγκρισης δύο αριθμών με πρόσημο
  - JL (s ≠ v)/JGE (s = v)
  - JG (z = 0 and s = v)/JLE (z = 1 or s ≠ v)
- Αποτέλεσμα σύγκρισης δύο αριθμών χωρίς πρόσημο
  - JB (c = 1)/JAE (c = 0)
  - JA (c = 0 and z = 0)/JBE (c = 1 or z = 1)

x86-64 (64-bit mode): Μορφή εντολών Jxx





Εάν ισχύει η συνθήκη xx (έλεγχος flags), τότε στον PC προστίθεται το offset (αριθμός με πρόσημο, 8 ή 32 bits)

x86-64 (64-bit mode): Εντολές διακλάδωσης χωρίς συνθήκη

- Σχετική διεύθυνση
  - PC ← PC + ακέραια σταθερά των 8 bits (μετατόπιση -128...+127 bytes) ή των 32 bits (μετατόπιση ±2GB)
- Απόλυτη διεύθυνση
  - PC ← 64 bits από έναν καταχωρητή ή θέση μνήμης (indirect)
- Διακλαδώσεις ειδικού σκοπού
  - Για μεταβάσεις του λειτουργικού συστήματος σε 64-bit ή 32-bit mode
  - Η διεύθυνση μετάβασης υπολογίζεται από πληροφορία αποθηκευμένη σε θέση μνήμης
  - Δεν θα ασχοληθούμε με αυτή την περίπτωση

Κλήσεις και επιστροφές από συναρτήσεις

- Είναι και αυτές είδος διακλάδωσης
  - Ο pc παίρνει ως νέα τιμή το σημείο εισόδου στη συνάρτηση ή επιστρέφει αμέσως μετά το σημείο κλήσης
- Ποια η διαφορά από τις «κανονικές» εντολές διακλάδωσης;
  - Δυνατότητα αποθήκευσης του αρχικού pc (πριν τη διακλάδωση)
    - Σε καταχωρητή ή σε θέση μνήμης (runtime stack)
  - Για να χρησιμοποιηθεί στο τέλος της συνάρτησης ως διεύθυνση επιστροφής (return address)

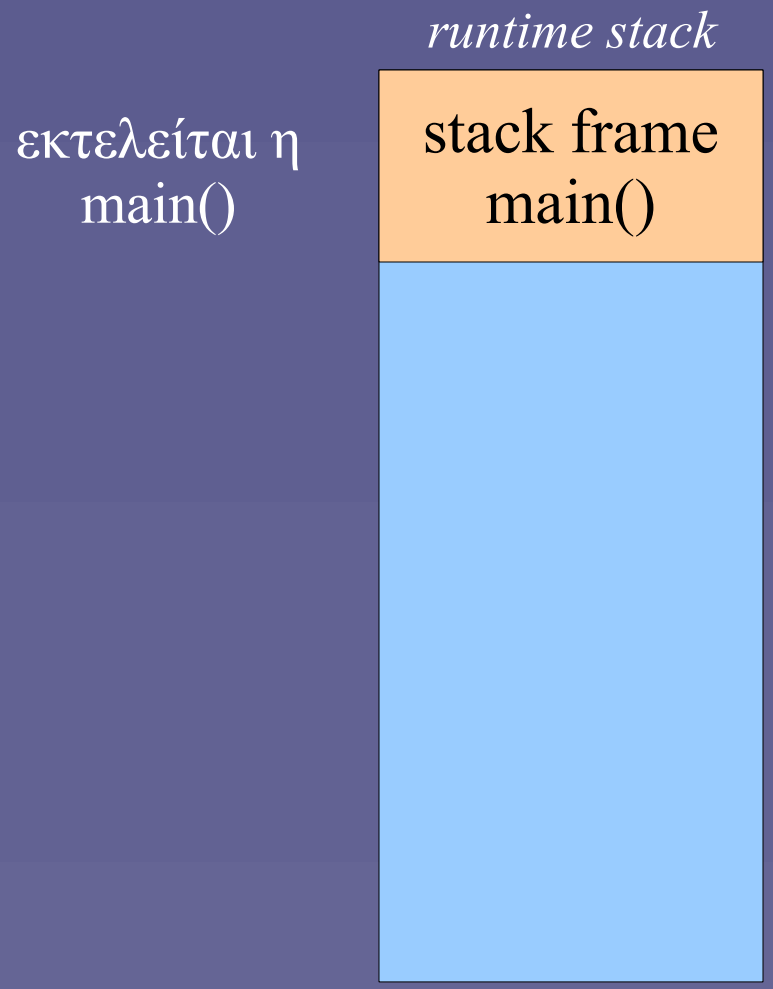
(Runtime) stack

- Ο χώρος στη μνήμη για την αποθήκευση
  - Των τοπικών μεταβλητών των συναρτήσεων
  - Της διεύθυνσης επιστροφής
  - Των παραμέτρων κλήσης της συνάρτησης
  - Των επιστρεφόμενων αποτελεσμάτων
  - Στις σύγχρονες αρχιτεκτονικές πολλά από τα παραπάνω παραμένουν όσο το δυνατόν στους καταχωρητές
- Stack pointer (sp)
  - Καταχωρητής γενικού ή ειδικού σκοπού (ανάλογα με την αρχιτεκτονική ISA) που περιέχει τη διεύθυνση της κορυφής στη στοίβα
    - Όπου θα εισαχθούν (push) ή θα αναγνωστούν (pop) δεδομένα
    - Η τιμή του προσαρμόζεται κατάλληλα μετά από κάθε push/pop

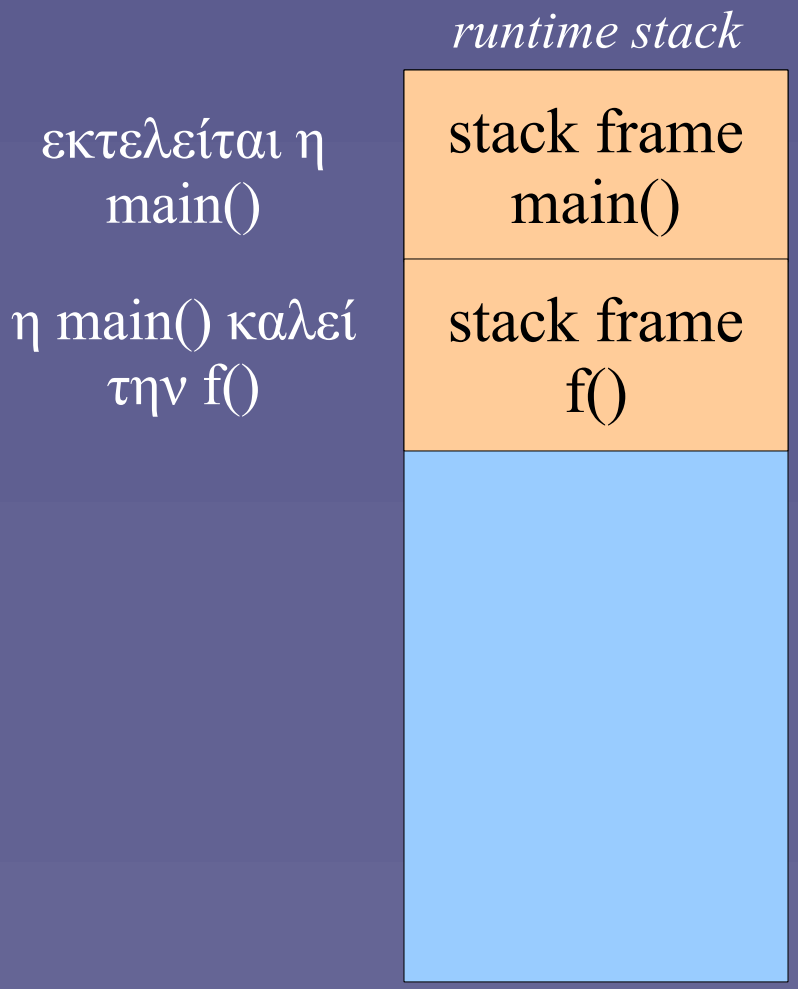
Stack Frame (ή Activation Record)

- Περιβάλλον εκτέλεσης μιας κλήσης συνάρτησης
  - Χώρος που δεσμεύεται στο runtime stack
  - Περιέχει το «περιβάλλον εκτέλεσης» μιας συνάρτησης
    - Τοπικές μεταβλητές
    - Παράμετροι εισόδου
    - Διεύθυνση επιστροφής
  - Κάθε κλήση συνάρτησης δημιουργεί ένα καινούργιο stack frame (αμέσως μετά το τρέχον stack frame)
- Frame pointer (fp)
  - Καταχωρητής (συνήθως γενικού σκοπού) που περιέχει τη διεύθυνση της αρχής του stack frame
  - Όλες οι προσπελάσεις σε τοπικές μεταβλητές γίνονται σε σχέση (±) με την τρέχουσα τιμή του fp

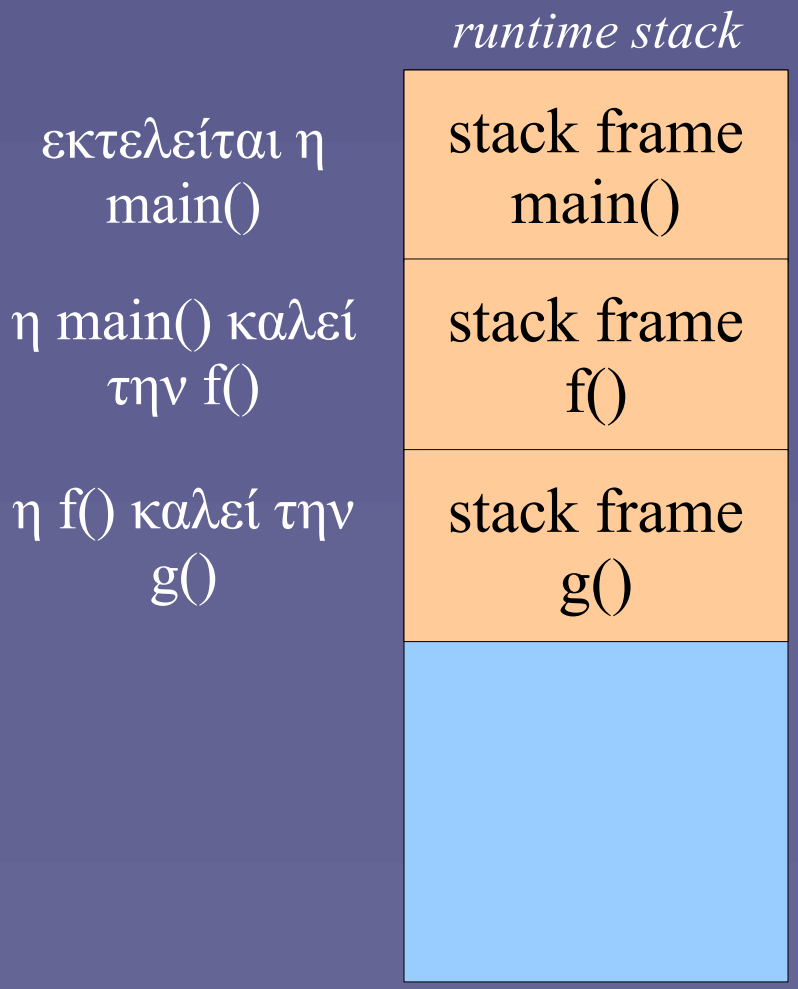
Stack frames (Activation records)



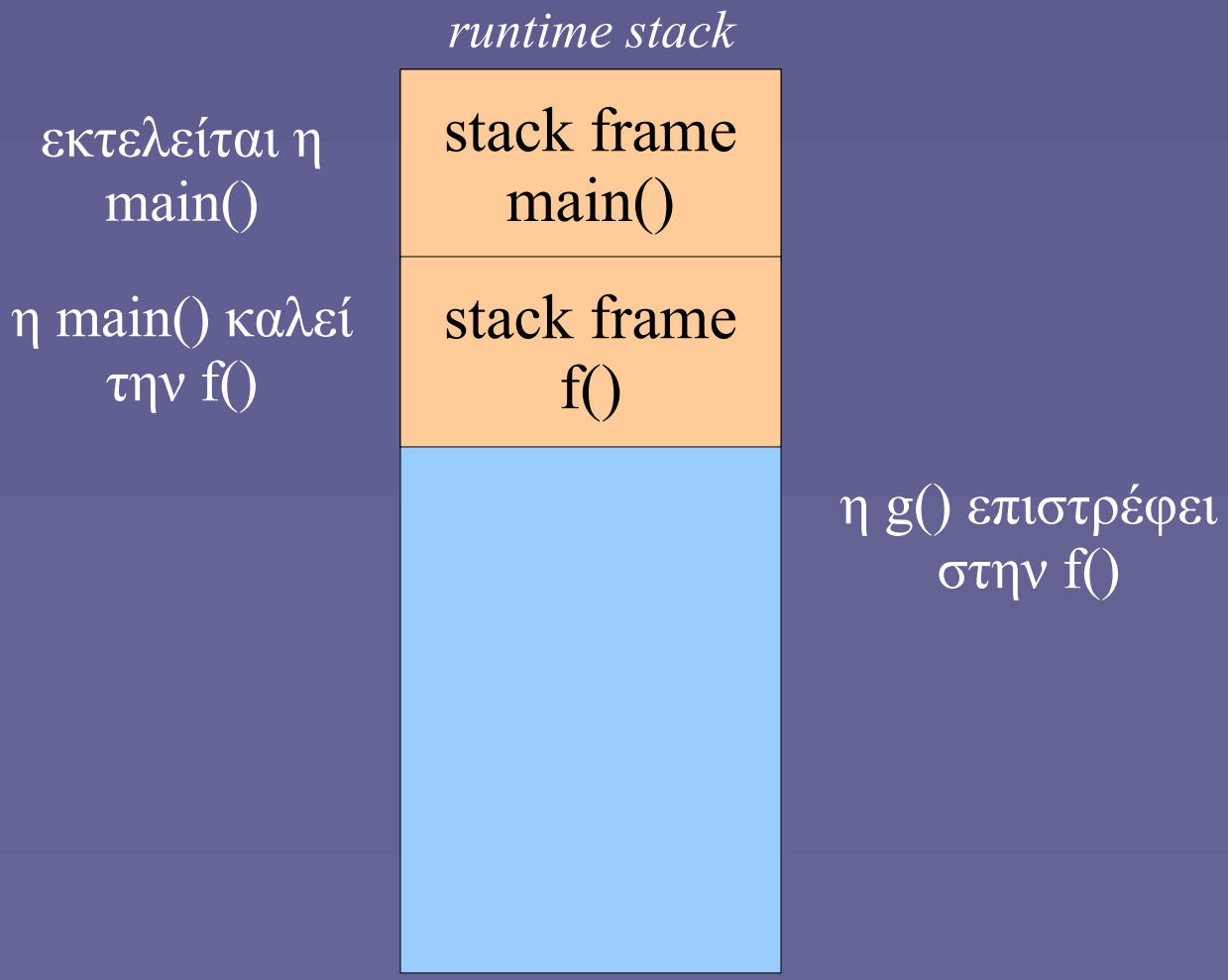
Stack frames (Activation records)



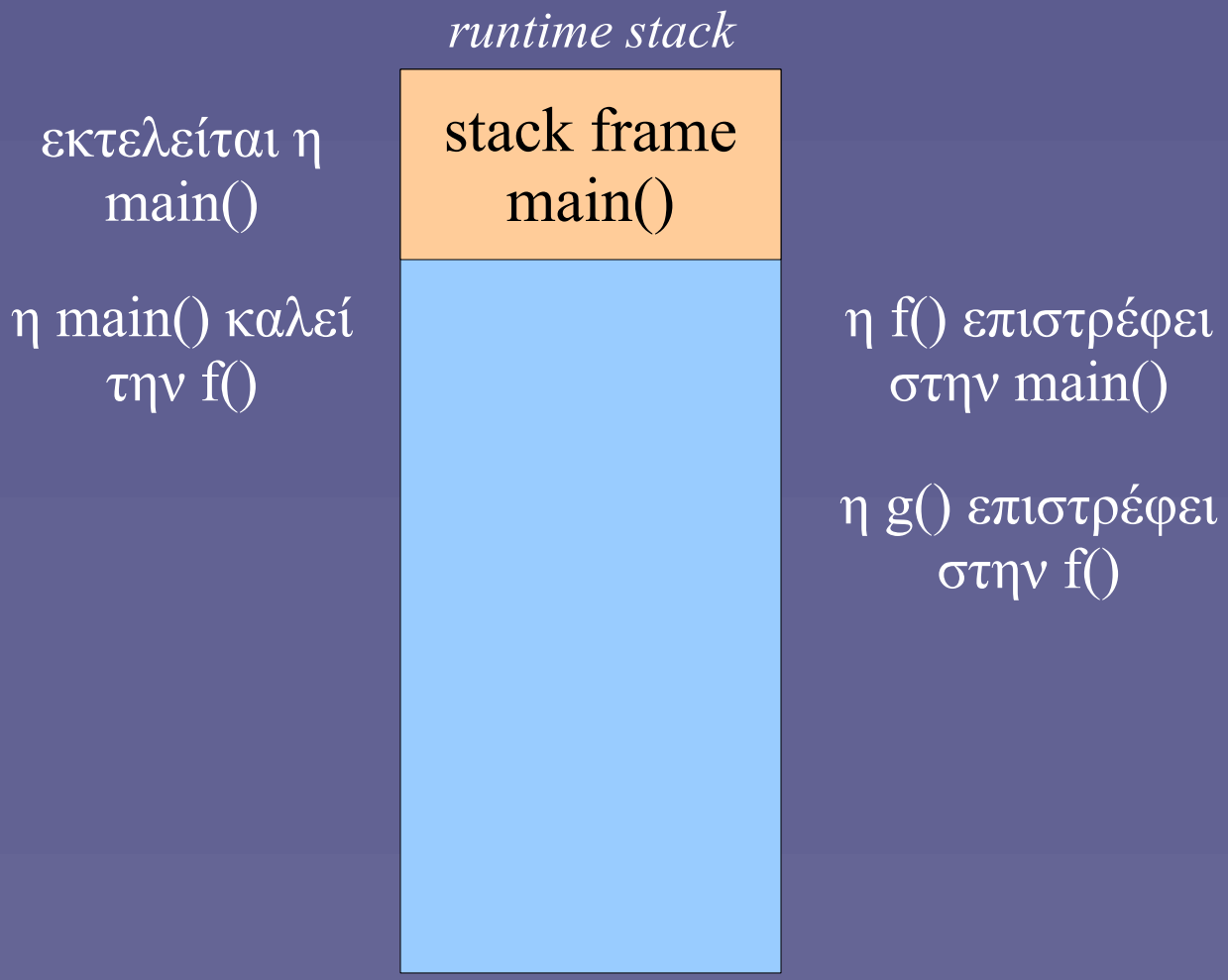
Stack frames (Activation records)



Stack frames (Activation records)



Stack frames (Activation records)



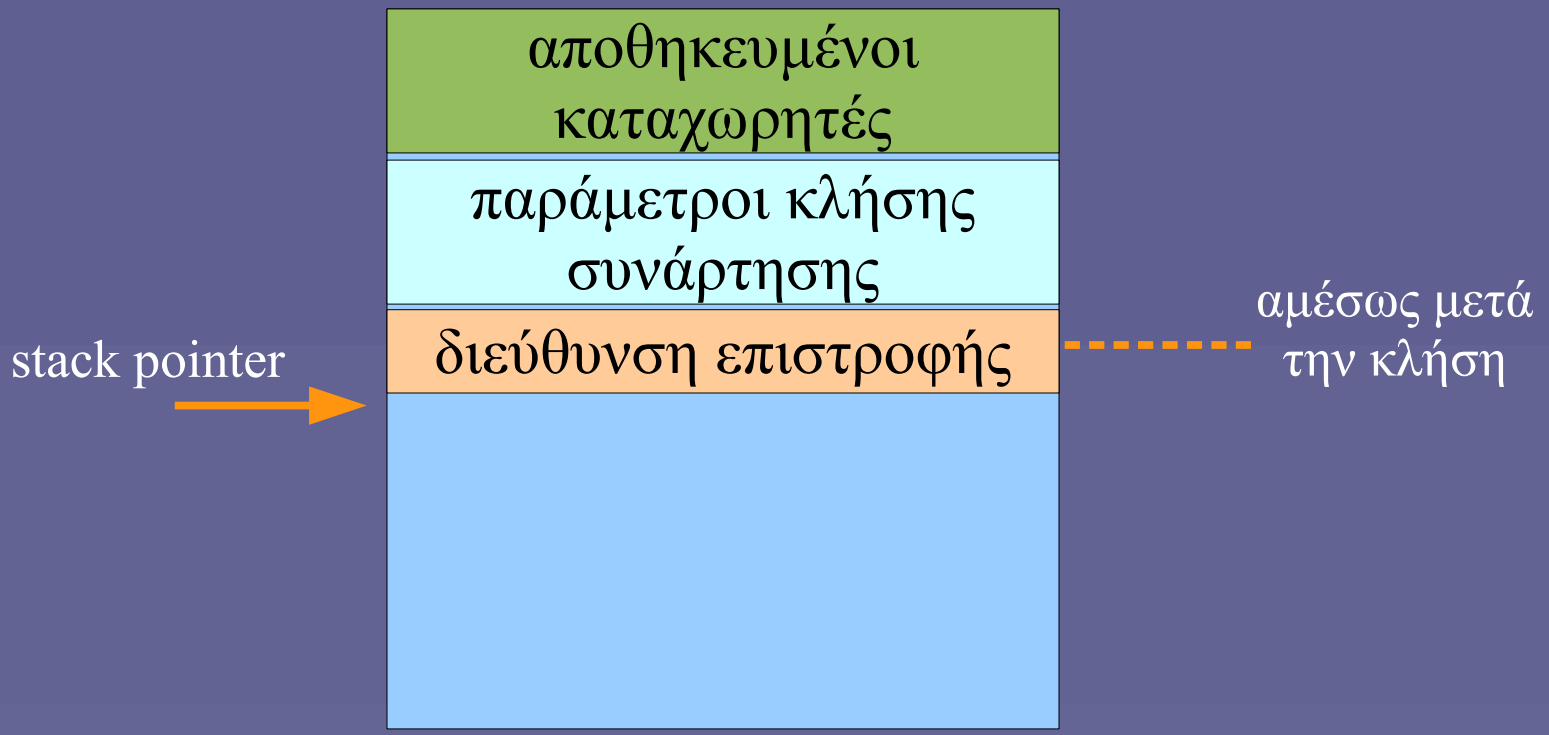
Γενική διαδικασία κλήσης συνάρτησης

- Η καλούσα συνάρτηση
  - Αποθηκεύει καταχωρητές που χρησιμοποιεί και πιθανόν να αλλάξει η καλούμενη συνάρτηση
  - Περνάει τις παραμέτρους κλήσης σε καταχωρητές ή στο stack frame
  - Χρησιμοποιεί εντολή μηχανής τύπου "call"
    - Διακλάδωση στην καλούμενη συνάρτηση
    - Με ταυτόχρονη αποθήκευση της διεύθυνσης επιστροφής (stack frame ή καταχωρητές)
- Μετά την επιστροφή
  - Αποκατάσταση αποθηκευμένων τιμών καταχωρητών

Γενική διαδικασία κλήσης συνάρτησης (2)

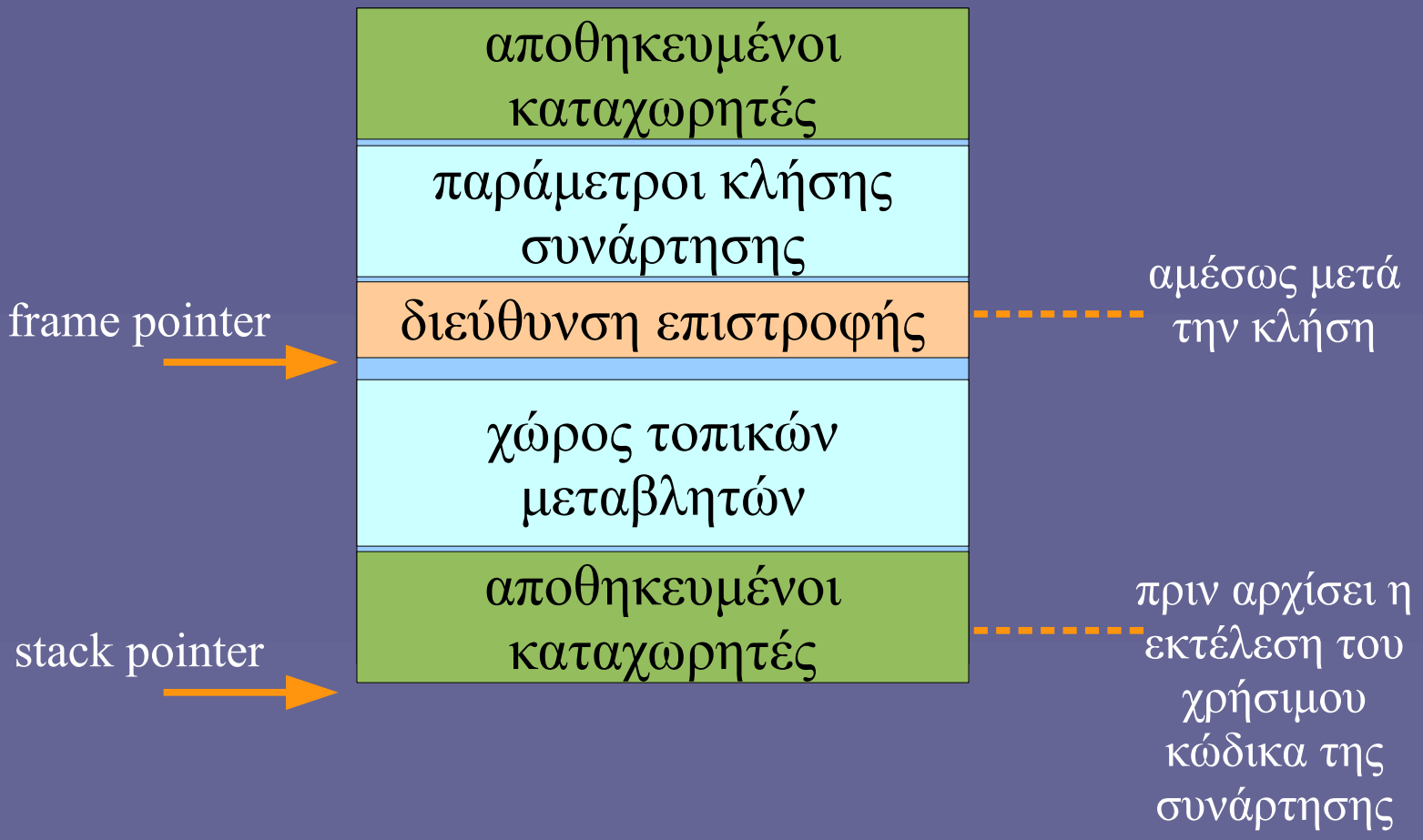
- Η καλούμενη συνάρτηση
  - Δημιουργεί χώρο για τις τοπικές μεταβλητές στο stack
  - Υπολογίζει τη νέα τιμή του frame pointer (fp) και αποθηκεύει την παλιά
  - Αποθηκεύει καταχωρητές που πιθανόν να αλλάξει
- Στη συνέχεια εκτελείται ο κώδικας της συνάρτησης
- Πριν το τέλος
  - Αποδέσμευση χώρου τοπικών μεταβλητών, αποκατάσταση τιμών καταχωρητών και αποθηκευμένου fp
  - Χρήση εντολής μηχανής τύπου “return”
    - Διακλάδωση στην αποθηκευμένη διεύθυνση επιστροφής

Παράδειγμα stack frame
(η συνάρτηση μόλις έχει κληθεί)



Παράδειγμα stack frame

(κατά την έναρξη εκτέλεσης του χρήσιμου κώδικα της συνάρτησης)



“Calling convention”

- Ο ακριβής τρόπος κλήσης των συναρτήσεων
  - Ποιες εντολές και ποιοι καταχωρητές χρησιμοποιούνται
  - Πού αποθηκεύονται τα ορίσματα της κλήσης
  - Η ακολουθία ενεργειών πριν και μετά την κλήση/επιστροφή
  - Πώς είναι η ακριβής μορφή του stack frame
- Εξαρτάται από την αρχιτεκτονική (ISA) και το λειτουργικό σύστημα
  - Application Binary Interface (ABI)

Συναρτήσεις και RISC-V ISA

- Δεν υπάρχουν ειδικοί καταχωρητές για τη στοίβα (stack) που να επιβάλλονται από το υλικό
  - Οι εντολές χρησιμοποιούν με ομοιόμορφο τρόπο όλους τους καταχωρητές
    - Δεν υπάρχουν εντολές που απαιτούν συγκεκριμένους μόνο καταχωρητές
- Κάποιοι καταχωρητές γενικού σκοπού χρησιμοποιούνται «κατά σύμβαση» στη διαδικασία κλήσης των συναρτήσεων
  - x1 → ra (return address)
  - x2 → sp (stack pointer)
  - x8 → fp (ή s0) (frame pointer)
  - x10-x17 → a0-a7 (argument registers) ορίσματα και επιστρεφόμενη τιμή (a0-a1)

Συναρτήσεις και RISC-V ISA

- «Κατά σύμβαση» η καλούμενη συνάρτηση πρέπει να διατηρήσει την τιμή ορισμένων καταχωρητών εάν πρόκειται να τους αλλάξει
  - Να τους αποθηκεύσει και να τους αποκαταστήσει στην αρχική τιμή πριν επιστρέψει
  - x2 (sp), x8 (fp), x9, x18-x27
  - Όλους τους υπόλοιπους καταχωρητές πρέπει να τους αποθηκεύσει η καλούσα συνάρτηση πριν καλέσει τη νέα συνάρτηση (αν θέλει να χρησιμοποιήσει τις τιμές τους μετά την κλήση)

RISC-V: Διαχείριση στοίβας (runtime stack)

- Η στοίβα «μεγαλώνει» προς χαμηλότερες διευθύνσεις
  - π.χ. addi sp, sp, -32 (δέσμευση χώρου για stack frame)
  - και addi sp, sp, 32 (αποδέσμευση χώρου stack frame πριν την επιστροφή)
- Η προσπέλαση των τοπικών μεταβλητών στο stack frame γίνεται με τη βοήθεια του fp (ή αλλιώς, s0)
  - π.χ. sw a1, -20(s0)
    - Λειτουργία: a1 → mem[s0 - 20]

RISC-V: Εντολές κλήσης και επιστροφής

- Κλήση συνάρτησης με εντολή jal (jump and link)
  - jal ra, funcaddr (ψευδοεντολή call funcaddr)
    - Λειτουργία: ra ← pc + 4 (next instruction), pc ← pc ± offset to funcaddr
- Επιστροφή με εντολή jalr (jump and link register)
  - jalr x0, ra, 0 (ψευδοεντολή jr ra ή ret)
    - Λειτουργία: x0 ← pc + 4 (next instruction), pc ← ra ± 0
    - Ο καταχωρητής x0 έχει ειδικό ρόλο: όταν τον διαβάζουμε δίνει πάντα 0 και ό,τι γράφεται σε αυτόν χάνεται
- Παράδειγμα: https://godbolt.org/z/vqb4Errbx

Συναρτήσεις και x86-64 ISA

- Χρησιμοποιείται συγκεκριμένος καταχωρητής για τη στοίβα
  - RSP, με εξειδικευμένες εντολές PUSH και POP, π.χ.
    - PUSH *src64*: rsp ← rsp – 8; mem[rsp] ← src64
    - POP *dst64*: dst64 ← mem[rsp]; rsp ← rsp + 8;
- Καταχωρητές γενικού σκοπού χρησιμοποιούνται στη διαδικασία κλήσης των συναρτήσεων (System V ABI)
  - rax, rdx → 1η, 2η επιστρεφόμενη τιμή (integer)
  - rdi, rsi, rdx, rcx, r8, r9 → ορίσματα συνάρτησης (integer, ptr)
  - rbp → χρησιμοποιείται ως frame pointer

Συναρτήσεις και x86-64 ISA

- «Κατά σύμβαση» η καλούμενη συνάρτηση πρέπει να διατηρήσει την τιμή ορισμένων καταχωρητών εάν πρόκειται να τους αλλάξει
  - Να τους αποθηκεύσει και να τους αποκαταστήσει στην αρχική τιμή πριν επιστρέψει
  - rbx, rsp, rbp, r12-r15
  - Όλους τους υπόλοιπους καταχωρητές πρέπει να τους αποθηκεύσει η καλούσα συνάρτηση πριν καλέσει τη νέα συνάρτηση (αν θέλει να χρησιμοποιήσει τις τιμές τους μετά την κλήση)

x86-64 (64-bit mode): Εντολές κλήσης και επιστροφής

- Κλήση συνάρτησης με εντολή call
  - Η διεύθυνση κλήσης μπορεί να υπολογίζεται σχετικά (relative, με 32-bit offset με πρόσημο) ως προς τον pc (RIP στην ορολογία της x86-64 ISA) που δείχνει στην επόμενη εντολή μετά το call
  - Ή να είναι μια 64-bit τιμή που περιέχεται σε έναν καταχωρητή ή μια θέση μνήμης (absolute, indirect)
  - Πριν την κλήση η τιμή του RIP αποθηκεύεται στη στοίβα
- Επιστροφή με εντολή ret
  - Εξαγωγή 64-bit (διεύθυνση επιστροφής) από τη στοίβα και ανάθεση στον RIP
- Παράδειγμα: https://godbolt.org/z/MxrbGq41a